Assessment Methodologies: Footprinting and Scanning

Capture the Flag (CTF 1) — Technical Lab Report

Platform	INE
Environment	Kali Linux (INE-provided lab environment)
Target	target.ine.local
Assessment type	Footprinting, service enumeration and information discovery
Author	Areeba Noor

1. Introduction

This report documents the approach, evidence and findings of Capture the Flag (CTF) 1 from the INE lab "Assessment Methodologies: Footprinting and Scanning". The lab provides GUI access to a Kali Linux machine, with the target available at **http://target.ine.local**. All activity was performed within the authorised lab environment.

Objective: perform reconnaissance on the target and capture the four flags hidden within the environment. Each flag comes with a clue, which was interpreted to select the most suitable tool and technique. Flag values are masked in all screenshots.

2. Flag Overview and Tools

Table 1: Lab clues and planned approach

Flag	Lab clue	Interpretation	Tool / technique
1	The server announces its identity in every response.	HTTP response headers may contain something unusual.	cURL
2	The gatekeeper's instructions reveal what should remain unseen.	The robots.txt file may list hidden paths.	cURL, browser
3	Anonymous access leads to forgotten treasures.	The FTP server may allow anonymous login.	Nmap, FTP
4	A well-named database is revealing; peek at the configurations.	Discovered files may hold database credentials.	FTP, MySQL

Table 2: Tools used

Tool	Purpose
cURL	Inspect HTTP response headers and website content
Nmap	Discover open ports and identify services
FTP client	Connect to the FTP service and retrieve files
MySQL client	Connect to the database server and enumerate databases

3. Initial Reconnaissance

A default Nmap scan was run first to map the services exposed by the target:

> **nmap target.ine.local**

The host was up and seven TCP ports were open (Figure 1): FTP (21), SSH (22), SMTP (25), HTTP (80), IMAP (143), IMAPS (993) and MySQL (3306). The HTTP, FTP and MySQL services align directly with the lab clues and became the focus of the assessment.

```
┌──(root㉿INE)-[~]
└─# nmap  target.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-24 07:33 IST
Nmap scan report for target.ine.local (192.107.25.3)
Host is up (0.000040s latency).
Not shown: 993 closed tcp ports (reset)
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
25/tcp   open  smtp
80/tcp   open  http
143/tcp  open  imap
993/tcp  open  imaps
3306/tcp open  mysql
MAC Address: 02:42:C0:6B:19:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.14 seconds
```

Figure 1: Default Nmap scan showing seven open TCP ports

4. Flag Walkthroughs

4.1 Flag 1: HTTP Header Disclosure

Clue: “The server proudly announces its identity in every response. Look closely; you might find something unusual.”

Step 1: Interpret the clue. A server that announces its identity in every response suggests that the HTTP response headers contain something unusual, so the headers were retrieved with cURL.

Step 2: Fetch the response headers using the following command:

```
curl -i http://target.ine.local
```

The **-i** option includes the HTTP response headers in the output. The uppercase **-I** option is an alternative that sends a HEAD request and returns the headers only, without the response body.

Step 3: Examine the response. It contained two Server headers: the first disclosed the web stack (Werkzeug/3.0.6 Python/3.10.12), and the second contained the first flag (Figure 2).

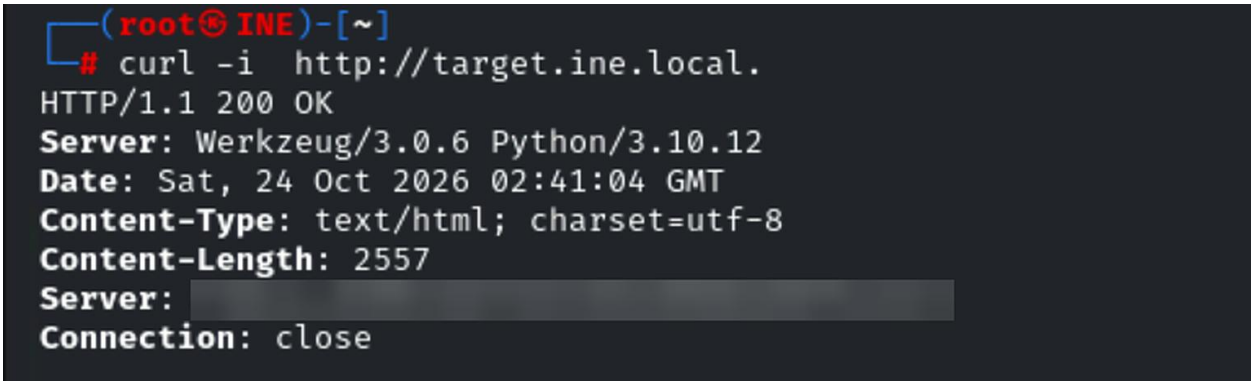


Figure 2: HTTP response headers returned by the target (flag value masked)

Key takeaway: response headers can reveal the technology behind a web application, and reviewing them is a quick and effective reconnaissance step. This disclosure helps an attacker fingerprint the target, although hiding version details is not a substitute for patching and securing the server.

4.2 Flag 2: Hidden Paths in robots.txt

Clue: "The gatekeeper's instructions often reveal what should remain unseen. Don't forget to read between the lines."

Step 1: Interpret the clue. The "gatekeeper" was taken to be robots.txt, the file that tells web crawlers which paths they should not visit.

Step 2: Request the file with the following command:

```
curl -i http://target.ine.local/robots.txt
```

Step 3: Examine the file. It disallowed /photos for Googlebot and /secret-info/ and /data/ for all other user agents (Figure 3).

```
┌──(root㉿INE)-[~]
└─# curl -i  http://target.ine.local/robots.txt
HTTP/1.1 200 OK
Server: Werkzeug/3.0.6 Python/3.10.12
Date: Sat, 24 Oct 2026 02:37:53 GMT
Content-Disposition: inline; filename=robots.txt
Content-Type: text/plain; charset=utf-8
Content-Length: 96
Last-Modified: Tue, 28 May 2024 05:55:46 GMT
Cache-Control: no-cache
ETag: "1716875746.0-96-1581123710"
Date: Sat, 24 Oct 2026 02:37:53 GMT
Server:
Connection: close

User-agent: googlebot
Disallow: /photos

User-agent: *
Disallow: /secret-info/
Disallow: /data/
```

Figure 3: Contents of robots.txt listing the disallowed paths (flag value masked)

Step 4: Visit the discovered path in the browser and capture the second flag.

Key takeaway: robots.txt only advises crawlers; it does not restrict access. Listing sensitive directories in it advertises their location to anyone who reads the file, so such content must be protected by proper authentication instead.

4.3 Flag 3: Anonymous FTP Access

Clue: "Anonymous access sometimes leads to forgotten treasures. Connect and explore the directory; you might stumble upon something valuable."

Step 1: Interpret the clue. FTP (File Transfer Protocol) allows clients to connect to a server and browse or transfer files, subject to the server's permissions. The reference to anonymous access suggested testing for anonymous login.

Step 2: Confirm that port 21 is open (Figure 1) and identify the service version with Nmap:

> **nmap -sV -p 21 target.ine.local**

The scan reported **vsftpd 3.0.5** running on a Unix system (Figure 4).

```
┌──(root㉿INE)-[~]
└─#  nmap -sV -p 21 target.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-10-24 07:33 IST
Nmap scan report for target.ine.local (192.107.25.3)
Host is up (0.000037s latency).

PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 3.0.5
MAC Address: 02:42:C0:6B:19:03 (Unknown)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.29 seconds
```

Figure 4: Nmap version scan identifying the FTP service on port 21

Step 3: Connect to the FTP service and log in with the username **anonymous**:

> **ftp target.ine.local**

Step 4: List the available files. The server accepted the login (230 Login successful), and the directory contained two world-readable files, **creds.txt** and **flag.txt** (Figure 5).

```
┌──(root㉿INE)-[~]
└─# ftp target.ine.local
Connected to target.ine.local.
220 (vsFTPd 3.0.5)
Name (target.ine.local:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> pwd
Remote directory: /
ftp> ls
229 Entering Extended Passive Mode (|||18785|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0              22 Oct 28  2024 creds.txt
-rw-r--r--    1 0        0              39 Oct 24 01:47 flag.txt
```

Figure 5: Anonymous FTP login and directory listing

Step 5: Download flag.txt with **get flag.txt** and read its contents to obtain the third flag (Figure 6).

```
ftp> ls la
229 Entering Extended Passive Mode (|||13387|)
150 Here comes the directory listing.
226 Directory send OK.
ftp> get flag.txt
local: flag.txt remote: flag.txt
229 Entering Extended Passive Mode (|||27467|)
150 Opening BINARY mode data connection for flag.txt (39 bytes).
100% |*****************************************************************************************************************|
226 Transfer complete.
39 bytes received in 00:00 (140.02 KiB/s)
```

Figure 6: Downloading flag.txt from the FTP server

Key takeaway: Nmap identified the service, and the FTP client allowed the directory to be explored and files to be retrieved. Anonymous FTP can expose files to unauthorised users when permissions are poorly configured, so unnecessary anonymous access should be disabled and file permissions restricted.

4.4 Flag 4: Credential Discovery and Database Enumeration

Clue: "A well-named database can be quite revealing. Peek at the configurations to discover the hidden treasure."

Step 1: Check that MySQL is reachable. Port 3306 was listed as open in the initial scan (Figure 1) and was confirmed with:

> **nmap -sV -p 3306 target.ine.local**

Step 2: Find the credentials. MySQL requires a username and password, so the FTP service was revisited. The second file, **creds.txt**, was downloaded with **get creds.txt** (Figure 7).

```
┌──(root㉿INE)-[~]
└─# ftp target.ine.local
Connected to target.ine.local.
220 (vsFTPd 3.0.5)
Name (target.ine.local:root): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> pwd
Remote directory: /
ftp> ls la
229 Entering Extended Passive Mode (|||28953|)
150 Here comes the directory listing.
226 Directory send OK.
ftp> ls
229 Entering Extended Passive Mode (|||13523|)
150 Here comes the directory listing.
-rw-r--r--    1 0        0              22 Oct 28  2024 creds.txt
-rw-r--r--    1 0        0              39 Oct 24 01:47 flag.txt
226 Directory send OK.
ftp> get creds.txt
local: creds.txt remote: creds.txt
229 Entering Extended Passive Mode (|||29821|)
150 Opening BINARY mode data connection for creds.txt (22 bytes).
100% |*****************************************************************************************************************|    22
226 Transfer complete.
```

Figure 7: Retrieving creds.txt through anonymous FTP

Step 3: Read the file with **cat creds.txt**. It contained the username and password of the database account **db_admin** (Figure 8).

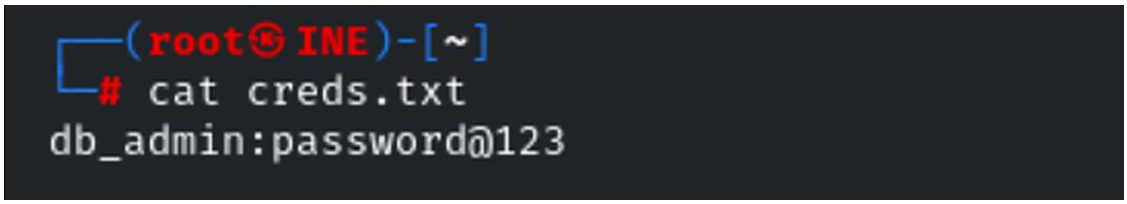


Figure 8: Contents of creds.txt

Step 4: Connect to the database server and enter the password when prompted:

> **mysql -h target.ine.local -u db_admin -p**

Here **-h** specifies the server hostname, **-u** the username, and **-p** prompts for the password.

Step 5: List the available databases. One database name contained the fourth flag (Figure 9).

> **SHOW DATABASES;**

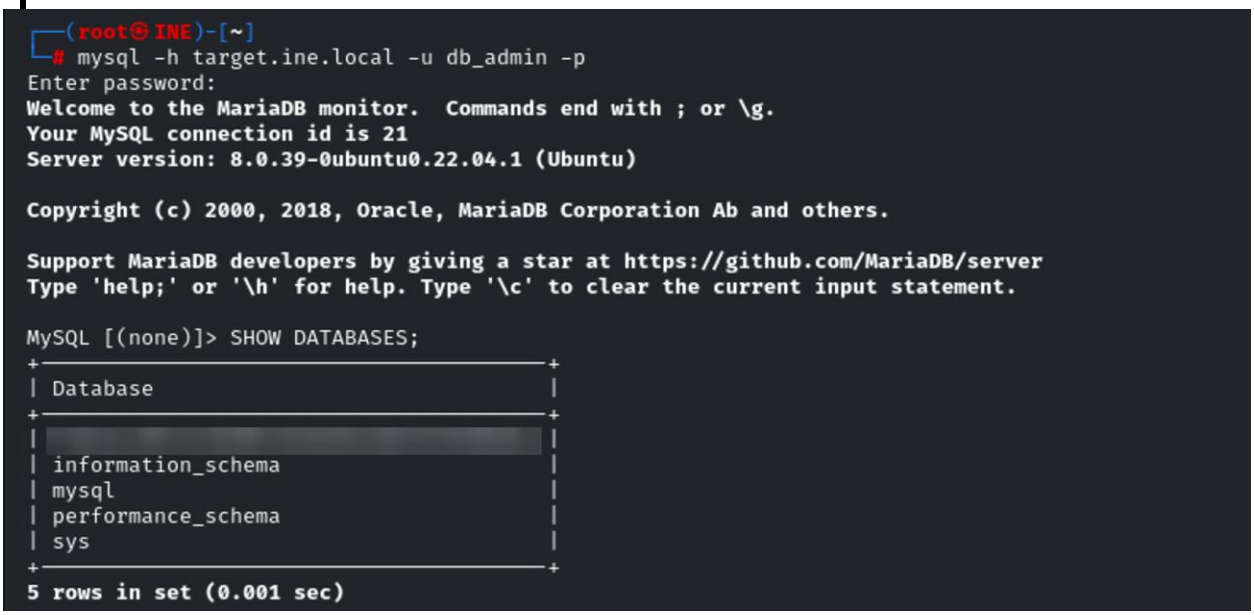


Figure 9: MySQL login and database listing (flag value masked)

Key takeaway: information found in one service can provide a lead into another. Here, FTP exposed a credentials file that enabled MySQL authentication. Credential security depends not only on password strength, but also on where credentials are stored, who can access them and what permissions the account holds.

5. Summary of Findings

Table 3 summarises the technique used for each flag, the weakness it exposed and the recommended control.

Table 3: Findings and recommended controls

Flag	Technique	Weakness identified	Recommended control
1	HTTP header inspection (cURL)	Server information disclosed in response headers	Remove or genericise unnecessary Server headers and keep software patched
2	robots.txt review (cURL, browser)	Sensitive paths listed in robots.txt	Do not rely on robots.txt for protection; enforce authentication on sensitive paths
3	Anonymous FTP login (Nmap, FTP)	Anonymous access to world-readable files	Disable anonymous FTP unless required; restrict permissions and exposed directories
4	Credential reuse (FTP, MySQL)	Plaintext credentials on FTP; MySQL reachable over the network	Remove and rotate exposed credentials; use strong passwords, least privilege and firewall rules for port 3306

6. Conclusion

All four flags were captured using a structured footprinting workflow: inspecting HTTP headers, reviewing robots.txt, scanning for services, and enumerating FTP and MySQL. The exercise showed how individually minor exposures — a verbose header, a revealing robots.txt, anonymous FTP and a plaintext credentials file — can be chained together to reach sensitive data.

It also reinforces the value of defence in depth: minimise information disclosure, restrict access to services and files, and protect credentials throughout their lifecycle.